

Lecture 4: Dynamic Programming

Given a recurrence, three ways to compute its values:

1.

Pro(s):

Con(s):

2.

Pro(s):

Con(s):

3.

Pro(s):

Con(s):

Which is/are efficient?

Weighted Task Selection Recurrence

Assume the intervals $J_1, J_2, \dots, J_n$ are sorted by _____ time.

Let OPT be an optimal solution. Start from J_n and reason backwards.

There are two options:

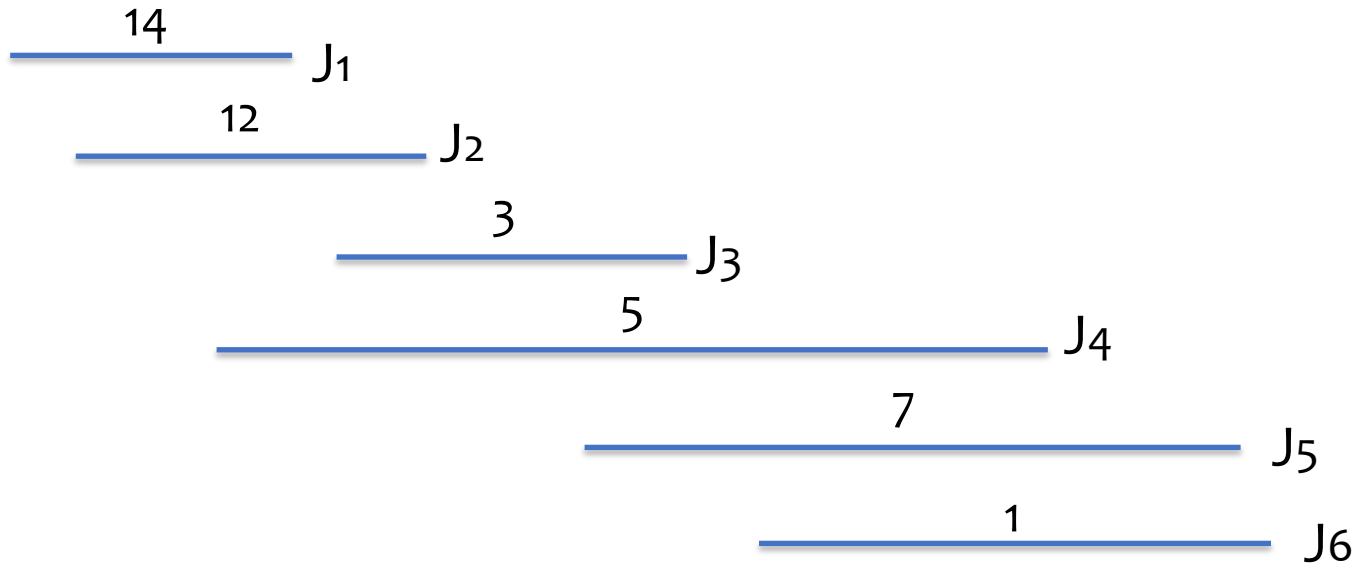
- OPT has J_n (use it!) $\leftarrow \text{OTV}(J_1, \dots, J_n) = \underline{\hspace{4cm}}$
- OPT doesn't have J_n (lose it!) $\leftarrow \text{OTV}(J_1, \dots, J_n) = \underline{\hspace{4cm}}$

Overall recurrence:

$\text{OTV}(J_1, \dots, J_n) =$

Dynamic Programming in Action!

OTV($\emptyset$)	OTV(J_1)	OTV(J_1, J_2)	OTV($J_1, \dots, J_3$)	OTV($J_1, \dots, J_4$)	OTV($J_1, \dots, J_5$)	OTV($J_1, \dots, J_6$)



usually the trickiest part

The DP Recipe

1. Define *value* version of what we want to compute
2. Derive a recurrence *“Let $OTV(J_1, \dots, J_n)$ be...”*
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?
6. Which cell(s) contain the final answer?
7. Running time is often (size of table) $\cdot$ (time to fill each entry)
8. To reconstruct the solution (instead of just its value) follow arrows from final answer to base case

Recurrence for Longest Increasing Subsequence

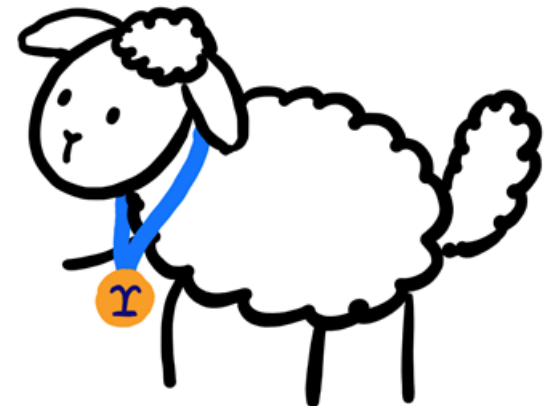
It's easy! Check it out!

$$\text{LIS}(A[1]) = 1$$

$$\text{LIS}(A[1..n]) = \begin{cases} \text{LIS}(A[1..n-1]) + 1 & \text{if } A[n] > A[n-1] \text{ (use it!)} \\ \text{LIS}(A[1..n-1]) & \text{otherwise (lose it!)} \end{cases}$$

Does this work?

Counterexample(?):



Recurrence for END-LIS

Defn: Let $\text{END-LIS}(A[1..i])$ be the length of an LIS of $A[1..i]$ *that ends in $A[i]$* .

What is $\text{END-LIS}(A[1..i])$ for each i in the previous example?

$A = [_, _, _, _, _, _, _, _, _, _, _],$

End-LIS:

Recurrence:

$\text{END-LIS}(A[1..i]) =$

Base case: $\text{END-LIS}(A[1..i]) =$

Let's follow the DP Recipe

END-LIS($A[1..i]$) = _____

A =	3	7	1	4	6	9	2	5
END-LIS($A[1..i]$)								

1. Define value version of what we want to compute
2. Derive a recurrence
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?
6. Which cell(s) contain the final answer?
7. Running time = (size of table) $\cdot$ (time to fill each entry)
8. To reconstruct the solution (instead of just its value) follow arrows from final answer to base case